

**LAPORAN PRAKTIKUM  
ALGORITMA & STRUKTUR DATA  
MODUL 6**



**GRAPH DAN TREE**

**Oleh:**

**Salma Nur Lathifah    NIM. 2510817120013**

**PROGRAM STUDI TEKNOLOGI INFORMASI  
FAKULTAS TEKNIK  
UNIVERSITAS LAMBUNG MANGKURAT  
JUNI  
2026**

**LEMBAR PENGESAHAN**  
**LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA**  
**MODUL 6**

Laporan Praktikum Algoritma & Struktur Data Modul 6: Graph dan Tree ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Salma Nur Lathifah  
NIM : 2510817120013

Menyetujui,  
Asisten Praktikum

Mengetahui,  
Dosen Penanggung Jawab Praktikum

Muhammad Fauzan Ahsani  
NIM. 2310817310009

Ir. Muhamamd Alkaff, S,Kom, M.Kom, Ph.D  
NIP. 198606132015041001

## DAFTAR ISI

|                                                           |     |
|-----------------------------------------------------------|-----|
| LEMBAR PENGESAHAN.....                                    | i   |
| DAFTAR ISI .....                                          | ii  |
| DAFTAR GAMBAR.....                                        | iii |
| DAFTAR TABEL .....                                        | iv  |
| SOAL 1: Konversi Adjacency List ke Adjacency Matrix ..... | 1   |
| A. Source Code.....                                       | 3   |
| B. Output Program .....                                   | 5   |
| C. Pembahasan .....                                       | 5   |
| SOAL 2: Konversi Adjacency Matrix ke Adjacency List.....  | 7   |
| A. Source Code.....                                       | 8   |
| B. Output Program .....                                   | 11  |
| C. Pembahasan .....                                       | 11  |
| SOAL 3: BFS: Jarak Terpendek Keluar Labirin .....         | 13  |
| A. Source Code.....                                       | 15  |
| B. Output Program .....                                   | 18  |
| C. Pembahasan .....                                       | 18  |
| SOAL 4: DFS: Menghitung Semua Jalur Keluar Labirin .....  | 20  |
| A. Source Code.....                                       | 22  |
| B. Output Program .....                                   | 25  |
| C. Pembahasan .....                                       | 25  |
| SOAL 5: Tree Traversal .....                              | 27  |
| A. Source Code.....                                       | 28  |
| B. Output Program .....                                   | 34  |
| C. Pembahasan .....                                       | 35  |
| TAUTAN GIT .....                                          | 36  |

## DAFTAR GAMBAR

|                                                 |    |
|-------------------------------------------------|----|
| Gambar 1. Screenshot Hasil Jawaban Soal 1 ..... | 5  |
| Gambar 2. Screenshot Hasil Jawaban Soal 2 ..... | 11 |
| Gambar 3. Screenshot Hasil Jawaban Soal 3 ..... | 18 |
| Gambar 4. Screenshot Hasil Jawaban Soal 4 ..... | 25 |
| Gambar 5. Screenshot Hasil Jawaban Soal 5 ..... | 34 |

## DAFTAR TABEL

|                                      |    |
|--------------------------------------|----|
| Tabel 1. Source Code soal1.cpp ..... | 3  |
| Tabel 2. Source Code soal2.cpp ..... | 8  |
| Tabel 3. Source Code soal3.cpp ..... | 15 |
| Tabel 4. Source Code soal4.cpp ..... | 22 |
| Tabel 5. Source Code soal5.cpp ..... | 28 |

## SOAL 1: Konversi Adjacency List ke Adjacency Matrix

Time Limit: 1.0 s  
Memory Limit: 64 MB

### Deskripsi Soal

Diberikan sebuah directed **weighted graph** yang terdiri dari  $N$  vertex dan  $M$  edge. Setiap vertex memiliki label berupa satu karakter unik. Graph diberikan dalam bentuk daftar edge.

Setiap edge ditulis sebagai  $U, V, W$ , yang berarti terdapat edge berarah dari vertex  $U$  menuju vertex  $V$  dengan bobot  $W$ . Karena graph bersifat berarah, edge  $U, V, W$  tidak otomatis berarti terdapat edge dari  $V$  menuju  $U$ .

Tugas Anda adalah mengubah representasi graph tersebut menjadi **adjacency matrix**. Jika terdapat edge dari vertex ke- $i$  menuju vertex ke- $j$ , maka nilai matrix pada baris  $i$  dan kolom  $j$  adalah bobot edge tersebut. Jika tidak terdapat edge, nilainya 0.

### Format Input

Input diberikan dalam format berikut.

$N$

$V_1 V_2 \dots V_N$

$M$

$U_1 V_1 W_1$

$U_2 V_2 W_2$

...

$U_M V_M W_M$

Keterangan:

- $N$  adalah jumlah vertex.
- $V_i$  adalah label vertex ke- $i$ .
- $M$  adalah jumlah edge.
- $U_i$  dan  $V_i$  adalah vertex asal dan vertex tujuan pada edge ke- $i$ .

- $W_i$  adalah bobot edge ke- $i$ .

### Format Output

Cetak adjacency matrix dengan format berikut.

Adjacency Matrix:

V1 V2 ... VN

V1 a11 a12 ... a1N

V2 a21 a22 ... a2N

...

VN aN1 aN2 ... aNN

Nilai  $a_{ij}$  adalah bobot edge dari vertex  $V_i$  menuju vertex  $V_j$ . Jika edge tersebut tidak ada, nilai  $a_{ij}$  adalah 0.

### Batasan

- $2 \leq N \leq 10$
- $0 \leq M \leq N * (N - 1)$
- $1 \leq W_i \leq 1000$
- Label vertex berupa karakter alfabet kapital A sampai Z.
- Semua label vertex dijamin unik.
- Graph tidak memiliki self-loop.
- Graph tidak memiliki edge duplikat dengan arah yang sama.
- Graph bersifat berarah dan berbobot.

### Contoh Kasus

| Input                                                                     | Output                                                                                                    |
|---------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| 5<br>A B C D E<br>6<br>A B 4<br>A C 2<br>B D 7<br>C B 1<br>C E 6<br>D C 3 | Adjacency Matrix:<br>A B C D E<br>A 0 4 2 0 0<br>B 0 0 0 7 0<br>C 0 1 0 0 6<br>D 0 0 3 0 0<br>E 0 0 0 0 0 |

## A. Source Code

*Tabel 1. Source Code soal1.cpp*

|    |                                                              |
|----|--------------------------------------------------------------|
| 1  | <code>#include &lt;iostream&gt;</code>                       |
| 2  | <code>#include &lt;vector&gt;</code>                         |
| 3  | <code>#include &lt;map&gt;</code>                            |
| 4  |                                                              |
| 5  | <code>using namespace std;</code>                            |
| 6  |                                                              |
| 7  | <code>int main() {</code>                                    |
| 8  | <code>int totalVertices;</code>                              |
| 9  | <code>cin &gt;&gt; totalVertices;</code>                     |
| 10 |                                                              |
| 11 | <code>vector&lt;char&gt; vertexLabels(totalVertices);</code> |
| 12 | <code>map&lt;char, int&gt; vertexIndex;</code>               |
| 13 |                                                              |
| 14 | <code>for (int i = 0; i &lt; totalVertices; i++) {</code>    |
| 15 | <code>cin &gt;&gt; vertexLabels[i];</code>                   |
| 16 | <code>vertexIndex[vertexLabels[i]] = i;</code>               |
| 17 | <code>}</code>                                               |
| 18 |                                                              |
| 19 | <code>int totalEdges;</code>                                 |
| 20 | <code>cin &gt;&gt; totalEdges;</code>                        |
| 21 |                                                              |



```

22     vector<vector<int>> adjacencyMatrix(
23         totalVertices,
24         vector<int>(totalVertices, 0)
25     );
26
27     for (int i = 0; i < totalEdges; i++) {
28         char sourceVertex;
29         char destinationVertex;
30         int edgeWeight;
31
32         cin >> sourceVertex >> destinationVertex >>
edgeWeight;
33
34         adjacencyMatrix[vertexIndex[sourceVertex]]
35             [vertexIndex[destinationVertex]]
36             = edgeWeight;
37     }
38
39     cout << "Adjacency Matrix:\n";
40
41     cout << " ";
42     for (int i = 0; i < totalVertices; i++) {
43         cout << " " << vertexLabels[i];
44     }
45     cout << '\n';
46
47     for (int i = 0; i < totalVertices; i++) {
48         cout << vertexLabels[i];
49
50         for (int j = 0; j < totalVertices; j++) {
51             cout << " " << adjacencyMatrix[i][j];

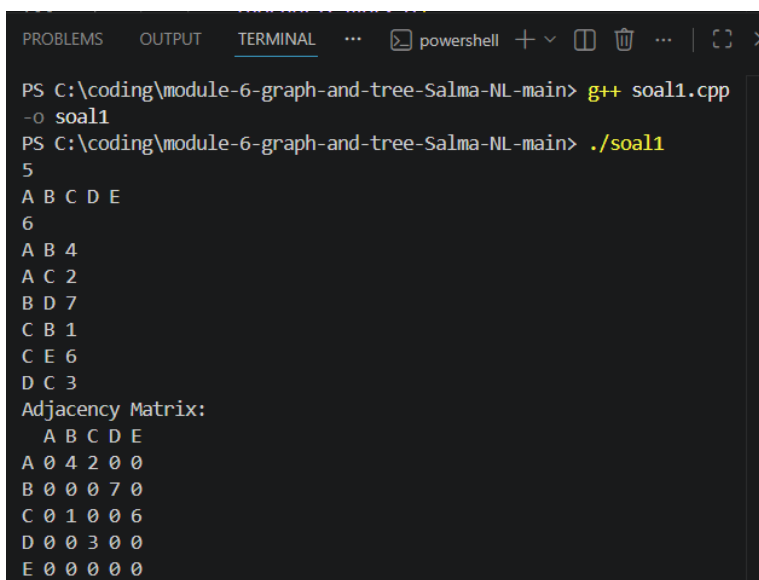
```

```

52     }
53
54     cout << '\n';
55 }
56
57 return 0;
58 }

```

## B. Output Program



```

PROBLEMS OUTPUT TERMINAL ... powershell + - [ ] [ ] ... | [ ] [ ] >
PS C:\coding\module-6-graph-and-tree-Salma-NL-main> g++ soal1.cpp
-o soal1
PS C:\coding\module-6-graph-and-tree-Salma-NL-main> ./soal1
5
A B C D E
6
A B 4
A C 2
B D 7
C B 1
C E 6
D C 3
Adjacency Matrix:
  A B C D E
A 0 4 2 0 0
B 0 0 0 7 0
C 0 1 0 0 6
D 0 0 3 0 0
E 0 0 0 0 0

```

Gambar 1. Screenshot Hasil Jawaban Soal 1

## C. Pembahasan

Algoritma pada soal 1 ini digunakan untuk mengubah representasi graph berarah berbobot dari bentuk adjacency list menjadi adjacency matrix. Teknik yang digunakan adalah memetakan setiap vertex ke indeks matriks, kemudian setiap edge ditempatkan pada posisi baris vertex asal dan kolom vertex tujuan. Dengan teknik tersebut, seluruh hubungan antar vertex dapat direpresentasikan dalam bentuk matriks berukuran  $N \times N$  agar lebih mudah untuk diakses dan dianalisis. Cara kerja algoritma dimulai dengan membaca seluruh vertex dan memetakan setiap vertex ke indeks matriks. Setelah adjacency matrix berukuran  $N \times N$

dibuat dengan nilai awal 0, setiap edge diproses satu per satu dan bobotnya ditempatkan pada posisi yang sesuai berdasarkan vertex asal dan tujuan.

Pada implementasinya diasumsikan bahwa setiap vertex memiliki label yang unik dan graph bersifat berarah. Setiap edge yang terdiri dari vertex asal, vertex tujuan, dan bobot diproses satu per satu, kemudian bobot tersebut ditempatkan pada posisi matriks yang sesuai berdasarkan indeks kedua vertex. Karena graph berarah, hubungan hanya dicatat pada arah yang diberikan oleh input dan tidak berlaku untuk arah sebaliknya.

Kompleksitas waktu algoritma ini adalah  $O(N^2 + M)$ , dengan  $N$  sebagai jumlah vertex dan  $M$  sebagai jumlah edge. Kompleksitas  $O(N^2)$  berasal dari proses pembuatan adjacency matrix, dan  $O(M)$  berasal dari proses pengisian edge ke dalam matriks. Sedangkan kompleksitas ruangnya adalah  $O(N^2)$  karena program menyimpan adjacency matrix berukuran  $N \times N$ . Pada contoh soal, edge  $A \rightarrow B$  dengan bobot 4 ditempatkan pada baris A kolom B dan edge  $C \rightarrow E$  dengan bobot 6 ditempatkan pada baris C kolom E. Setelah seluruh edge diproses, adjacency matrix yang dihasilkan merepresentasikan graph sesuai data input sehingga dapat disimpulkan bahwa algoritma berhasil melakukan konversi adjacency list menjadi adjacency matrix dengan benar.

## SOAL 2: Konversi Adjacency Matrix ke Adjacency List

Time Limit: 1.0 s  
Memory Limit: 64 MB

### Deskripsi Soal

Diberikan sebuah directed **weighted graph** yang direpresentasikan dalam bentuk **adjacency matrix** berukuran  $N \times N$ . Setiap vertex memiliki label berupa satu karakter unik.

Nilai matrix pada baris  $i$  dan kolom  $j$  menunjukkan bobot edge dari vertex ke- $i$  menuju vertex ke- $j$ . Jika nilainya 0, berarti tidak terdapat edge dari vertex ke- $i$  menuju vertex ke- $j$ .

Tugas Anda adalah mengubah adjacency matrix tersebut menjadi **adjacency list**. Untuk setiap vertex, cetak semua vertex tujuan yang dapat dicapai langsung beserta bobot edge-nya. Urutan vertex tujuan mengikuti urutan label pada input.

### Format Input

Input diberikan dalam format berikut.

$N$

$V_1 V_2 \dots V_N$

$A_{11} A_{12} \dots A_{1N}$

$A_{21} A_{22} \dots A_{2N}$

...

$A_{N1} A_{N2} \dots A_{NN}$

Keterangan:

- $N$  adalah jumlah vertex.
- $V_i$  adalah label vertex ke- $i$ .
- $A_{ij}$  adalah nilai matrix pada baris ke- $i$  dan kolom ke- $j$ .
- Jika  $A_{ij} > 0$ , maka terdapat edge dari  $V_i$  menuju  $V_j$  dengan bobot  $A_{ij}$ .
- Jika  $A_{ij} = 0$ , maka tidak terdapat edge dari  $V_i$  menuju  $V_j$ .

## Format Output

Cetak adjacency list untuk setiap vertex dengan format berikut.

Adjacency List:

V1 -> (X1,w1), (X2,w2), ...

V2 -> (X1,w1), (X2,w2), ...

...

VN -> ...

Jika sebuah vertex tidak memiliki edge keluar, cetak tanda - setelah simbol ->.

## Constraints

- $2 \leq N \leq 10$
- $0 \leq A_{ij} \leq 1000$
- Label vertex berupa karakter alfabet kapital A sampai Z.
- Semua label vertex dijamin unik.
- Nilai diagonal utama matrix dijamin 0.
- Nilai 0 berarti tidak ada edge.
- Nilai positif berarti bobot edge.
- Matrix merepresentasikan graph berarah dan berbobot.

## Contoh Kasus

| Input                                                                           | Output                                                                                          |
|---------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| 5<br>A B C D E<br>0 4 2 0 0<br>0 0 0 7 0<br>0 1 0 0 6<br>0 0 3 0 0<br>0 0 0 0 0 | Adjacency List:<br>A -> (B,4), (C,2)<br>B -> (D,7)<br>C -> (B,1), (E,6)<br>D -> (C,3)<br>E -> - |

## A. Source Code

Tabel 2. Source Code soal2.cpp

|   |                     |
|---|---------------------|
| 1 | #include <iostream> |
|---|---------------------|

```

2  #include <vector>
3
4  using namespace std;
5
6  int main() {
7      int totalVertices;
8      cin >> totalVertices;
9
10     vector<char> vertexLabels(totalVertices);
11
12     for (int i = 0; i < totalVertices; i++) {
13         cin >> vertexLabels[i];
14     }
15
16     vector<vector<int>> adjacencyMatrix(
17         totalVertices,
18         vector<int>(totalVertices)
19     );
20
21     for (int i = 0; i < totalVertices; i++) {
22         for (int j = 0; j < totalVertices; j++) {
23             cin >> adjacencyMatrix[i][j];
24         }
25     }
26
27     cout << "Adjacency List:\n";
28
29     for (int i = 0; i < totalVertices; i++) {
30
31         cout << vertexLabels[i] << " -> ";
32

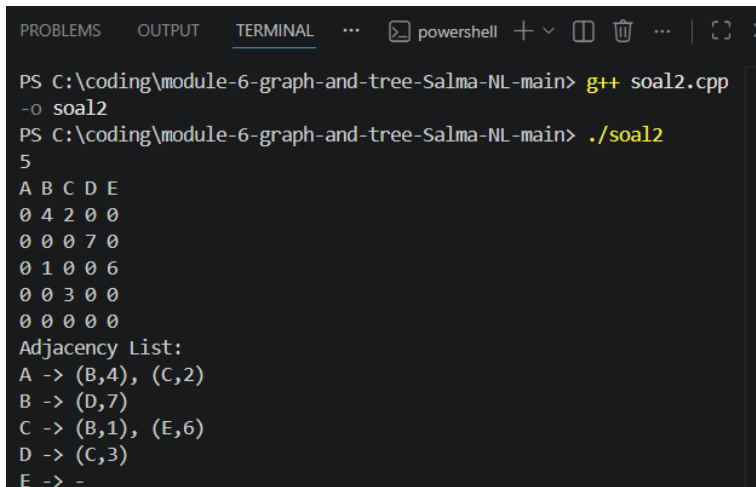
```

```

33         bool hasOutgoingEdge = false;
34
35         for (int j = 0; j < totalVertices; j++) {
36
37             if (adjacencyMatrix[i][j] > 0) {
38
39                 if (hasOutgoingEdge) {
40                     cout << ", ";
41                 }
42
43                 cout << "("
44                     << vertexLabels[j]
45                     << ", "
46                     << adjacencyMatrix[i][j]
47                     << ") ";
48
49                 hasOutgoingEdge = true;
50             }
51         }
52
53         if (!hasOutgoingEdge) {
54             cout << "-";
55         }
56
57         cout << '\n';
58     }
59
60     return 0;
61 }

```

## B. Output Program



```
PROBLEMS OUTPUT TERMINAL ... powershell + v [ ] [ ] ... | [ ] [ ] x
PS C:\coding\module-6-graph-and-tree-Salma-NL-main> g++ soal2.cpp
-o soal2
PS C:\coding\module-6-graph-and-tree-Salma-NL-main> ./soal2
5
A B C D E
0 4 2 0 0
0 0 0 7 0
0 1 0 0 6
0 0 3 0 0
0 0 0 0 0
Adjacency List:
A -> (B,4), (C,2)
B -> (D,7)
C -> (B,1), (E,6)
D -> (C,3)
E -> -
```

Gambar 2. Screenshot Hasil Jawaban Soal 2

## C. Pembahasan

Pada soal 2, algoritma digunakan untuk mengubah representasi graph berarah berbobot dari bentuk adjacency matrix menjadi adjacency list. Teknik yang digunakan adalah melakukan penelusuran pada setiap elemen matriks dan mencatat seluruh nilai yang menunjukkan adanya hubungan antar vertex. Jadi dengan teknik tersebut, graph yang awalnya direpresentasikan dalam bentuk matriks dapat diubah menjadi daftar hubungan yang lebih ringkas.

Dalam implementasinya diasumsikan bahwa nilai 0 menunjukkan tidak adanya edge, sedangkan nilai selain 0 menunjukkan bobot edge dari suatu vertex ke vertex lainnya. Program membaca seluruh isi adjacency matrix, kemudian melakukan iterasi pada setiap baris dan kolom. Jika ditemukan nilai yang lebih besar dari 0, maka vertex tujuan beserta bobotnya akan ditambahkan ke adjacency list dari vertex asal.

Kompleksitas waktu algoritma ini adalah  $O(N^2)$  karena seluruh elemen pada adjacency matrix harus diperiksa satu per satu. Dan kompleksitas ruangnya adalah  $O(N^2)$  karena program menyimpan adjacency matrix berukuran  $N \times N$ . Pada contoh soal, nilai 4 pada baris A kolom B akan menghasilkan pasangan (B,4) pada adjacency list milik A, sedangkan nilai 7 pada baris B kolom D akan menghasilkan pasangan (D,7) pada adjacency list milik B. Setelah seluruh matriks diproses, adjacency list yang dihasilkan



merepresentasikan graph yang sama dengan representasi awal sehingga algoritma dapat dikatakan bekerja dengan benar.

### SOAL 3: BFS: Jarak Terpendek Keluar Labirin

Time Limit: 1.0 s

Memory Limit: 64 MB

#### Deskripsi Soal

Sebuah labirin direpresentasikan sebagai grid berukuran  $R \times C$ . Setiap sel pada grid memiliki nilai sebagai berikut.

- 0 menyatakan jalan yang dapat dilewati.
- 1 menyatakan dinding yang tidak dapat dilewati.

Diberikan posisi awal dan posisi tujuan. Anda harus menentukan jumlah langkah minimum dari posisi awal menuju posisi tujuan menggunakan algoritma **Breadth-First Search** (BFS).

Dari sebuah sel, Anda hanya boleh bergerak ke empat arah: atas, bawah, kiri, dan kanan. Pergerakan diagonal tidak diperbolehkan. Beberapa jalur pada labirin dapat berupa jalan buntu, sehingga algoritma harus tetap memproses kemungkinan jalur secara sistematis.

#### Format Input

Input diberikan dalam format berikut.

$R$   $C$

$G_{11}$   $G_{12}$  ...  $G_{1C}$

$G_{21}$   $G_{22}$  ...  $G_{2C}$

...

$G_{R1}$   $G_{R2}$  ...  $G_{RC}$

$S_R$   $S_C$

$F_R$   $F_C$

Keterangan:

- $R$  adalah jumlah baris grid.
- $C$  adalah jumlah kolom grid.
- $G_{ij}$  adalah nilai sel pada baris ke- $i$  dan kolom ke- $j$ .

- $(S_R, S_C)$  adalah posisi awal.
- $(F_R, F_C)$  adalah posisi tujuan.

Indeks baris dan kolom dimulai dari 0.

### **Format Output**

Cetak satu bilangan bulat.

$X$

Keterangan:

- $X$  adalah jumlah langkah minimum dari posisi awal ke posisi tujuan.
- Jika posisi tujuan tidak dapat dicapai, cetak  $-1$ .

### **Constraints**

- $1 \leq R \leq 100$
- $1 \leq C \leq 100$
- $G_{ij}$  hanya bernilai 0 atau 1.
- $0 \leq S_R < R$
- $0 \leq S_C < C$
- $0 \leq F_R < R$
- $0 \leq F_C < C$
- Sel start dan finish dijamin bernilai 0.
- Pergerakan hanya boleh dilakukan ke sel bernilai 0.
- Algoritma yang digunakan wajib BFS.

### **Contoh Kasus**

| Input                                                                                                                                                                                     | Output |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|
| <pre> 8 10 0 0 0 0 1 0 0 0 0 0 1 1 1 0 1 0 1 1 1 0 0 0 1 0 0 0 0 0 1 0 0 1 1 1 1 1 1 0 1 0 0 0 0 0 0 0 1 0 0 0 1 1 1 1 1 0 1 1 1 0 0 0 0 0 1 0 0 0 1 0 0 1 1 0 0 0 1 0 0 0 0 0 7 9 </pre> | 16     |

## A. Source Code

*Tabel 3. Source Code soal3.cpp*

|    |                                                                |
|----|----------------------------------------------------------------|
| 1  | <code>#include &lt;iostream&gt;</code>                         |
| 2  | <code>#include &lt;vector&gt;</code>                           |
| 3  | <code>#include &lt;queue&gt;</code>                            |
| 4  |                                                                |
| 5  | <code>using namespace std;</code>                              |
| 6  |                                                                |
| 7  | <code>struct Position {</code>                                 |
| 8  | <code>    int row;</code>                                      |
| 9  | <code>    int column;</code>                                   |
| 10 | <code>};</code>                                                |
| 11 |                                                                |
| 12 | <code>int main() {</code>                                      |
| 13 | <code>    int totalRows;</code>                                |
| 14 | <code>    int totalColumns;</code>                             |
| 15 |                                                                |
| 16 | <code>    cin &gt;&gt; totalRows &gt;&gt; totalColumns;</code> |
| 17 |                                                                |
| 18 | <code>    vector&lt;vector&lt;int&gt;&gt; mazeGrid(</code>     |
| 19 | <code>        totalRows,</code>                                |
| 20 | <code>        vector&lt;int&gt;(totalColumns)</code>           |

```

21     );
22
23     for (int i = 0; i < totalRows; i++) {
24         for (int j = 0; j < totalColumns; j++) {
25             cin >> mazeGrid[i][j];
26         }
27     }
28
29     int startRow;
30     int startColumn;
31
32     int finishRow;
33     int finishColumn;
34
35     cin >> startRow >> startColumn;
36     cin >> finishRow >> finishColumn;
37
38     vector<vector<int>> minimumDistance(
39         totalRows,
40         vector<int>(totalColumns, -1)
41     );
42
43     queue<Position> bfsQueue;
44
45     bfsQueue.push({startRow, startColumn});
46     minimumDistance[startRow][startColumn] = 0;
47
48     int rowDirection[4] = {-1, 1, 0, 0};
49     int columnDirection[4] = {0, 0, -1, 1};
50
51     while (!bfsQueue.empty()) {

```

```

52
53     Position currentPosition = bfsQueue.front();
54     bfsQueue.pop();
55
56     for (int k = 0; k < 4; k++) {
57
58         int nextRow =
59             currentPosition.row + rowDirection[k];
60
61         int nextColumn =
62             currentPosition.column +
63             columnDirection[k];
64
65         if (nextRow < 0 || nextRow >= totalRows ||
66             nextColumn < 0 || nextColumn >=
67             totalColumns) {
68             continue;
69         }
70
71         if (mazeGrid[nextRow][nextColumn] == 1) {
72             continue;
73         }
74
75         if (minimumDistance[nextRow][nextColumn] != -
76             1) {
77             continue;
78         }
79
80         minimumDistance[nextRow][nextColumn] =
81             minimumDistance[currentPosition.row]

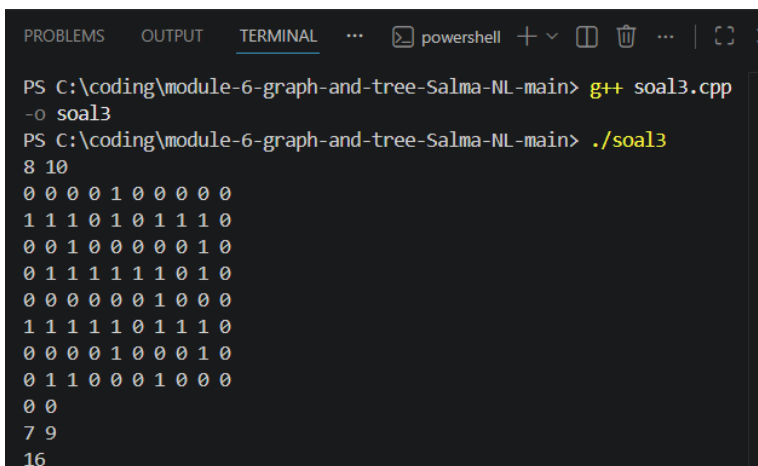
```

```

79         [currentPosition.column] +
80         1;
81         bfsQueue.push({nextRow, nextColumn});
82     }
83 }
84
85 cout << minimumDistance[finishRow][finishColumn] <<
86 '\n';
87
88 return 0;
89 }

```

## B. Output Program



```

PS C:\coding\module-6-graph-and-tree-Salma-NL-main> g++ soal3.cpp
-o soal3
PS C:\coding\module-6-graph-and-tree-Salma-NL-main> ./soal3
8 10
0 0 0 0 1 0 0 0 0 0
1 1 1 0 1 0 1 1 1 0
0 0 1 0 0 0 0 0 1 0
0 1 1 1 1 1 1 0 1 0
0 0 0 0 0 0 1 0 0 0
1 1 1 1 1 0 1 1 1 0
0 0 0 0 1 0 0 0 1 0
0 1 1 0 0 0 1 0 0 0
0 0
7 9
16

```

Gambar 3. Screenshot Hasil Jawaban Soal 3

## C. Pembahasan

Algoritma pada program soal 3 menggunakan Breadth First Search (BFS) untuk mencari jarak terpendek dari posisi awal menuju posisi tujuan pada labirin. Teknik BFS dipilih karena mampu menelusuri setiap posisi secara bertahap berdasarkan tingkat kedekatannya dari titik awal sehingga jalur pertama yang mencapai tujuan merupakan jalur terpendek. Cara kerja algoritma dimulai dari titik awal yang dimasukkan ke dalam queue,

kemudian setiap sel yang berdekatan diperiksa dan diberi nilai jarak dari titik awal hingga posisi tujuan ditemukan.

Pada implementasi program ini, diasumsikan bahwa sel bernilai 0 dapat dilalui dan sel bernilai 1 merupakan penghalang. Setiap sel yang telah dikunjungi tidak akan diproses kembali agar tidak terjadi pengulangan pencarian. Kompleksitas waktu algoritma adalah  $O(R \times C)$ , dengan  $R$  sebagai jumlah baris dan  $C$  sebagai jumlah kolom labirin. Kompleksitas ruangnya juga  $O(R \times C)$  karena program menyimpan matriks jarak dan queue yang dapat berisi banyak sel.

Berdasarkan contoh soal, BFS akan menjelajahi seluruh jalur yang memungkinkan dari titik awal hingga mencapai tujuan dan menghasilkan jumlah langkah minimum yang diperlukan. Oleh karena itu, algoritma BFS sesuai digunakan untuk menyelesaikan permasalahan pencarian jarak terpendek pada labirin.



## SOAL 4: DFS: Menghitung Semua Jalur Keluar Labirin

Time Limit: 2.0 s  
Memory Limit: 64 MB

### Deskripsi Soal

Diberikan sebuah labirin berukuran  $R \times C$  dengan aturan sel yang sama seperti soal sebelumnya. Anda harus menghitung banyaknya jalur sederhana dari posisi awal menuju posisi tujuan menggunakan algoritma **Depth-First Search** (DFS) rekursif.

Sebuah jalur disebut sederhana jika tidak mengunjungi sel yang sama lebih dari satu kali dalam jalur yang sama. Dari sebuah sel, Anda hanya boleh bergerak ke empat arah: atas, bawah, kiri, dan kanan.

Labirin dapat memiliki beberapa percabangan dan jalan buntu. Karena itu, DFS harus melakukan proses **backtracking** agar dapat mencoba semua kemungkinan jalur yang valid.

### Format Input

Input diberikan dalam format berikut.

$R$   $C$

$G_{11}$   $G_{12}$  ...  $G_{1C}$

$G_{21}$   $G_{22}$  ...  $G_{2C}$

...

$G_{R1}$   $G_{R2}$  ...  $G_{RC}$

$S_R$   $S_C$

$F_R$   $F_C$

Keterangan:

- $R$  adalah jumlah baris grid.
- $C$  adalah jumlah kolom grid.
- $G_{ij}$  adalah nilai sel pada baris ke- $i$  dan kolom ke- $j$ .
- $(S_R, S_C)$  adalah posisi awal.

- $(F_R, F_C)$  adalah posisi tujuan.

Indeks baris dan kolom dimulai dari 0.

### Format Output

Cetak satu bilangan bulat.

T

Keterangan:

- T adalah jumlah jalur sederhana dari posisi awal ke posisi tujuan.
- Jika tidak ada jalur yang dapat dilalui, cetak 0.

### Constraints

- $1 \leq R \leq 7$
- $1 \leq C \leq 7$
- $G_{ij}$  hanya bernilai 0 atau 1.
- $0 \leq S_R < R$
- $0 \leq S_C < C$
- $0 \leq F_R < R$
- $0 \leq F_C < C$
- Sel start dan finish dijamin bernilai 0.
- Pergerakan hanya boleh dilakukan ke sel bernilai 0.
- Satu jalur tidak boleh mengunjungi sel yang sama lebih dari satu kali.
- Jumlah jalur valid pada test case dijamin tidak lebih dari 100000.
- Algoritma yang digunakan wajib DFS rekursif dengan backtracking

### Contoh Kasus

| Input                                                                                | Output |
|--------------------------------------------------------------------------------------|--------|
| <pre> 5 6 0 0 0 1 0 0 0 1 0 0 0 1 0 1 0 1 0 0 0 0 0 1 1 0 1 1 0 0 0 0 0 0 4 5 </pre> | 4      |

## A. Source Code

*Tabel 4. Source Code soal4.cpp*

|    |                                                         |
|----|---------------------------------------------------------|
| 1  | #include <iostream>                                     |
| 2  | #include <vector>                                       |
| 3  |                                                         |
| 4  | using namespace std;                                    |
| 5  |                                                         |
| 6  | int totalRows;                                          |
| 7  | int totalColumns;                                       |
| 8  |                                                         |
| 9  | int finishRow;                                          |
| 10 | int finishColumn;                                       |
| 11 |                                                         |
| 12 | long long totalValidPaths = 0;                          |
| 13 |                                                         |
| 14 | vector<vector<int>> mazeGrid;                           |
| 15 | vector<vector<bool>> visitedCells;                      |
| 16 |                                                         |
| 17 | int rowDirection[4] = {-1, 1, 0, 0};                    |
| 18 | int columnDirection[4] = {0, 0, -1, 1};                 |
| 19 |                                                         |
| 20 | void countPathsDFS(int currentRow, int currentColumn) { |
| 21 |                                                         |
| 22 | if (currentRow == finishRow &&                          |
| 23 | currentColumn == finishColumn) {                        |
| 24 |                                                         |
| 25 | totalValidPaths++;                                      |

```

26         return;
27     }
28
29     visitedCells[currentRow][currentColumn] = true;
30
31     for (int k = 0; k < 4; k++) {
32
33         int nextRow =
34             currentRow + rowDirection[k];
35
36         int nextColumn =
37             currentColumn + columnDirection[k];
38
39         if (nextRow < 0 || nextRow >= totalRows ||
40             nextColumn < 0 || nextColumn >= totalColumns)
41         {
42             continue;
43         }
44
45         if (mazeGrid[nextRow][nextColumn] == 1) {
46             continue;
47         }
48
49         if (visitedCells[nextRow][nextColumn]) {
50             continue;
51         }
52
53         countPathsDFS(nextRow, nextColumn);
54     }
55
56     visitedCells[currentRow][currentColumn] = false;

```

```

56 }
57
58 int main() {
59
60     cin >> totalRows >> totalColumns;
61
62     mazeGrid.assign(
63         totalRows,
64         vector<int>(totalColumns)
65     );
66
67     for (int i = 0; i < totalRows; i++) {
68         for (int j = 0; j < totalColumns; j++) {
69             cin >> mazeGrid[i][j];
70         }
71     }
72
73     int startRow;
74     int startColumn;
75
76     cin >> startRow >> startColumn;
77     cin >> finishRow >> finishColumn;
78
79     visitedCells.assign(
80         totalRows,
81         vector<bool>(totalColumns, false)
82     );
83
84     countPathsDFS(startRow, startColumn);
85
86     cout << totalValidPaths << '\n';

```

|    |           |
|----|-----------|
| 87 |           |
| 88 | return 0; |
| 89 | }         |

## B. Output Program

```

PROBLEMS OUTPUT TERMINAL ... powershell + - 
PS C:\coding\module-6-graph-and-tree-Salma-NL-main> g++ soal4.cpp
-o soal4
PS C:\coding\module-6-graph-and-tree-Salma-NL-main> ./soal4
5 6
0 0 0 1 0 0
0 1 0 0 0 1
0 1 0 1 0 0
0 0 0 1 1 0
1 1 0 0 0 0
0 0
4 5
4

```

Gambar 4. Screenshot Hasil Jawaban Soal 4

## C. Pembahasan

Pada soal 4 ini, digunakan algoritma Depth First Search (DFS) untuk menghitung seluruh kemungkinan jalur dari posisi awal menuju posisi tujuan pada labirin. Teknik DFS dipilih karena mampu menelusuri satu jalur hingga mencapai titik akhir sebelum kembali melakukan pencarian pada jalur lainnya melalui proses backtracking. Cara kerja algoritma dimulai dari posisi awal dengan menelusuri setiap jalur yang memungkinkan menggunakan rekursi. Ketika mencapai tujuan atau tidak ditemukan lagi jalur yang dapat dilalui, algoritma akan melakukan backtracking untuk kembali ke posisi sebelumnya dan mencoba jalur lain hingga seluruh kemungkinan jalur selesai diperiksa.

Algoritma ini mengasumsikan bahwa sel bernilai 0 dapat dilalui dan sel bernilai 1 merupakan penghalang. Program memulai pencarian dari posisi awal kemudian menelusuri seluruh kemungkinan arah yang tersedia. Setiap sel yang sedang dikunjungi akan ditandai sebagai visited agar tidak membentuk siklus, kemudian status tersebut akan dikembalikan saat proses backtracking sehingga jalur lain masih dapat diperiksa.

Kompleksitas waktu algoritma ini bergantung pada jumlah kemungkinan jalur yang dapat dibentuk dari posisi awal menuju posisi tujuan. Pada worst case, algoritma harus menelusuri seluruh jalur yang memungkinkan sehingga waktu eksekusinya dapat meningkat secara signifikan dengan kompleksitas mendekati  $O(4^{R \times C})$ . Kompleksitas ruangnya adalah  $O(R \times C)$  karena program menyimpan matriks visited dan menggunakan rekursi DFS selama proses pencarian. Berdasarkan contoh soal, DFS akan mencoba seluruh kombinasi jalur yang mungkin dari titik awal menuju tujuan dan menambah penghitung setiap kali tujuan berhasil dicapai. Oleh karena itu, algoritma DFS mampu menghitung seluruh jalur yang valid sesuai dengan ketentuan soal.

## SOAL 5: Tree Traversal

Time Limit: 1.0 s  
Memory Limit: 64 MB

### Deskripsi Soal

Diberikan  $N$  bilangan bulat. Masukkan seluruh bilangan tersebut ke dalam sebuah **Red Black Tree** secara berurutan sesuai input. Jika terdapat bilangan duplikat, bilangan tersebut diabaikan dan tidak dimasukkan kembali ke tree.

Setelah RBTree terbentuk, diberikan  $Q$  query. Setiap query meminta salah satu hasil traversal berikut.

- PREORDER, yaitu urutan root, subtree kiri, lalu subtree kanan.
- INORDER, yaitu urutan subtree kiri, root, lalu subtree kanan.
- POSTORDER, yaitu urutan subtree kiri, subtree kanan, lalu root.
- ALL, yaitu mencetak ketiga traversal sekaligus.

Tugas Anda adalah mencetak hasil traversal sesuai query yang diberikan.

### Format Input

Input diberikan dalam format berikut.

$N$

$A_1 A_2 \dots A_N$

$Q$

$T_1$

$T_2$

...

$T_Q$

Keterangan:

- $N$  adalah banyaknya bilangan yang akan dimasukkan ke RBTree.
- $A_i$  adalah bilangan ke- $i$ .



- Q adalah banyaknya query traversal.
- $T_i$  adalah jenis traversal yang diminta.

### Format Output

Untuk setiap query, cetak hasil traversal sesuai format berikut.

[Preorder] : daftar\_angka

[Inorder] : daftar\_angka

[Postorder] : daftar\_angka

Jika query adalah ALL, cetak ketiga baris traversal dengan urutan Preorder, Inorder, lalu Postorder.

Jika setelah penghapusan duplikat tree tetap kosong, cetak:

Tree kosong. Tidak ada yang bisa ditampilkan.

### Constraints

- $0 \leq N \leq 1000$
- $-10000000000 \leq A_i \leq 10000000000$
- $1 \leq Q \leq 20$
- $T_i$  hanya salah satu dari PREORDER, INORDER, POSTORDER, atau ALL.

### Contoh Kasus

| Input                                                                     | Output                                                                                                                                                                                                                     |
|---------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 7<br>50 30 70 20 40 60 80<br>4<br>PREORDER<br>INORDER<br>POSTORDER<br>ALL | [Preorder] : 50 30 20 40 70 60 80<br>[Inorder] : 20 30 40 50 60 70 80<br>[Postorder] : 20 40 30 60 80 70 50<br>[Preorder] : 50 30 20 40 70 60 80<br>[Inorder] : 20 30 40 50 60 70 80<br>[Postorder] : 20 40 30 60 80 70 50 |

### A. Source Code

Tabel 5. Source Code soal5.cpp

|   |                     |
|---|---------------------|
| 1 | #include <iostream> |
|---|---------------------|

```

2  #include <vector>
3  #include <string>
4  #include "RedBlackTree.h"
5
6  using namespace std;
7
8  void preorderTraversal(const RedBlackTree::Node*
currentNode,
9                          const RedBlackTree::Node*
nilNode,
10                         vector<int>& traversalResult) {
11      if (currentNode == nilNode) return;
12
13      traversalResult.push_back(currentNode->key);
14      preorderTraversal(currentNode->left, nilNode,
traversalResult);
15      preorderTraversal(currentNode->right, nilNode,
traversalResult);
16  }
17
18  void inorderTraversal(const RedBlackTree::Node*
currentNode,
19                      const RedBlackTree::Node* nilNode,
20                      vector<int>& traversalResult) {
21      if (currentNode == nilNode) return;
22
23      inorderTraversal(currentNode->left, nilNode,
traversalResult);
24      traversalResult.push_back(currentNode->key);
25      inorderTraversal(currentNode->right, nilNode,
traversalResult);

```

```

26 }
27
28 void postorderTraversal(const RedBlackTree::Node*
currentNode,
29 const RedBlackTree::Node*
nilNode,
30 vector<int>& traversalResult) {
31     if (currentNode == nilNode) return;
32
33     postorderTraversal(currentNode->left, nilNode,
traversalResult);
34     postorderTraversal(currentNode->right, nilNode,
traversalResult);
35     traversalResult.push_back(currentNode->key);
36 }
37
38 int main() {
39     ios::sync_with_stdio(false);
40     cin.tie(nullptr);
41
42     int totalNumbers;
43     cin >> totalNumbers;
44
45     RedBlackTree redBlackTree;
46
47     for (int i = 0; i < totalNumbers; i++) {
48         int number;
49         cin >> number;
50
51         if (!redBlackTree.contains(number)) {
52             redBlackTree.insert(number);

```

```

53         }
54     }
55
56     int totalQueries;
57     cin >> totalQueries;
58
59     if (redBlackTree.empty()) {
60         cout << "Tree kosong. Tidak ada yang bisa
ditampilkan.\n";
61         return 0;
62     }
63
64     for (int i = 0; i < totalQueries; i++) {
65         string traversalType;
66         cin >> traversalType;
67
68         vector<int> preorderResult;
69         vector<int> inorderResult;
70         vector<int> postorderResult;
71
72         if (traversalType == "PREORDER") {
73             preorderTraversal(redBlackTree.root(),
74                             redBlackTree.nil(),
75                             preorderResult);
76
77             cout << "[Preorder] : ";
78             for (size_t j = 0; j < preorderResult.size();
j++) {
79                 cout << preorderResult[j];
80                 if (j + 1 < preorderResult.size()) cout
<< " ";

```

```

81         }
82         cout << '\n';
83     }
84
85     else if (traversalType == "INORDER") {
86         inorderTraversal(redBlackTree.root(),
87                         redBlackTree.nil(),
88                         inorderResult);
89
90         cout << "[Inorder] : ";
91         for (size_t j = 0; j < inorderResult.size();
92             j++) {
93             cout << inorderResult[j];
94             if (j + 1 < inorderResult.size()) cout <<
95             " ";
96         }
97         cout << '\n';
98     }
99
100    else if (traversalType == "POSTORDER") {
101        postorderTraversal(redBlackTree.root(),
102                          redBlackTree.nil(),
103                          postorderResult);
104
105        cout << "[Postorder] : ";
106        for (size_t j = 0; j <
107            postorderResult.size(); j++) {
108            cout << postorderResult[j];
109            if (j + 1 < postorderResult.size()) cout
110            << " ";
111        }

```

```

108         cout << '\n';
109     }
110
111     else if (traversalType == "ALL") {
112         preorderTraversal(redBlackTree.root(),
113                         redBlackTree.nil(),
114                         preorderResult);
115
116         inorderTraversal(redBlackTree.root(),
117                         redBlackTree.nil(),
118                         inorderResult);
119
120         postorderTraversal(redBlackTree.root(),
121                         redBlackTree.nil(),
122                         postorderResult);
123
124         cout << "[Preorder] : ";
125         for (size_t j = 0; j < preorderResult.size();
126             j++) {
127             cout << preorderResult[j];
128             if (j + 1 < preorderResult.size()) cout
129             << " ";
130         }
131         cout << '\n';
132
133         cout << "[Inorder] : ";
134         for (size_t j = 0; j < inorderResult.size();
135             j++) {
136             cout << inorderResult[j];
137             if (j + 1 < inorderResult.size()) cout <<
138             " ";
139         }
140         cout << '\n';
141     }
142 }

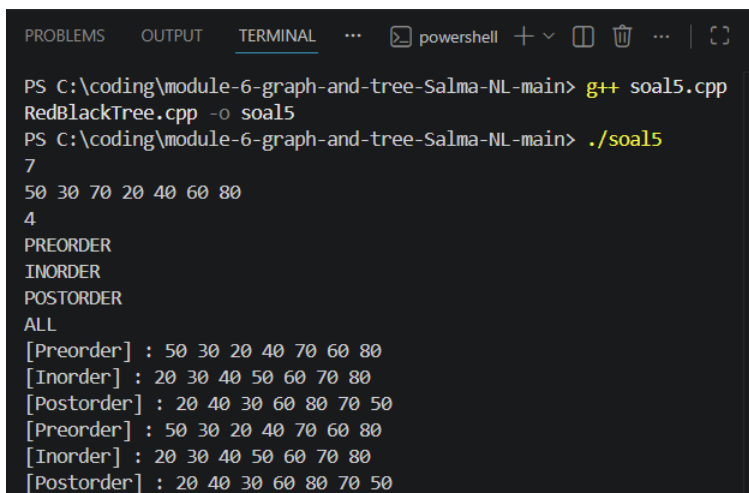
```

```

135         }
136         cout << '\n';
137
138         cout << "[Postorder] : ";
139         for (size_t j = 0; j <
postorderResult.size(); j++) {
140             cout << postorderResult[j];
141             if (j + 1 < postorderResult.size()) cout
<< " ";
142         }
143         cout << '\n';
144     }
145 }
146
147 return 0;
148 }

```

## B. Output Program



```

PROBLEMS OUTPUT TERMINAL ... powershell + - 
PS C:\coding\module-6-graph-and-tree-Salma-NL-main> g++ soal5.cpp
RedBlackTree.cpp -o soal5
PS C:\coding\module-6-graph-and-tree-Salma-NL-main> ./soal5
7
50 30 70 20 40 60 80
4
PREORDER
INORDER
POSTORDER
ALL
[Preorder] : 50 30 20 40 70 60 80
[Inorder] : 20 30 40 50 60 70 80
[Postorder] : 20 40 30 60 80 70 50
[Preorder] : 50 30 20 40 70 60 80
[Inorder] : 20 30 40 50 60 70 80
[Postorder] : 20 40 30 60 80 70 50

```

Gambar 5. Screenshot Hasil Jawaban Soal 5

### C. Pembahasan

Pada soal 5 ini, hasil traversal pada Red-Black Tree ditampilkan menggunakan metode preorder, inorder, dan postorder. Teknik yang digunakan adalah traversal rekursif, yaitu mengunjungi setiap node berdasarkan aturan traversal yang dipilih. Struktur Red-Black Tree digunakan karena merupakan binary search tree yang menjaga keseimbangan tinggi tree sehingga operasi pencarian dan penyisipan tetap efisien. Cara kerja algoritma dimulai dengan memasukkan seluruh data ke dalam Red-Black Tree, kemudian program menjalankan traversal sesuai query yang diberikan.

Dalam penerapannya, diasumsikan bahwa data yang dimasukkan berupa bilangan unik sehingga nilai duplikat tidak dimasukkan kembali ke dalam tree. Setelah seluruh data tersimpan, program akan menjalankan traversal sesuai query yang diberikan. Traversal preorder mengunjungi root terlebih dahulu, inorder mengunjungi subtree kiri kemudian root dan subtree kanan, sedangkan postorder mengunjungi kedua subtree sebelum root.

Kompleksitas waktu untuk setiap proses traversal adalah  $O(N)$ , dengan  $N$  sebagai jumlah node dalam tree, karena setiap node dikunjungi tepat satu kali. Kompleksitas ruangnya adalah  $O(N)$  karena program menyimpan hasil traversal dan menggunakan rekursi selama proses penelusuran tree. Pada contoh soal, data dimasukkan ke dalam tree kemudian ditampilkan menggunakan preorder, inorder, atau postorder sesuai permintaan pengguna. Hasil traversal yang diperoleh merepresentasikan struktur tree yang terbentuk sehingga algoritma berhasil menampilkan urutan kunjungan node sesuai jenis traversal yang dipilih.



## TAUTAN GIT

<https://github.com/DSA26-ULM/module-6-graph-and-tree-Salma-NL.git>